Assignment 6 report

Team: Music Band | Group: SE-2501

Kanatov Sabyrzhan and Zhaxylyk Alikhan

Objective: Continue the Cold Carti fan website with jQuery search, feedback and loading interactions.

Task 0. Setup

1. Load jQuery 4.0.0 from the CDN in the head and start script.js after the document is ready.
2. Check the console message: jQuery is ready! Keep the same local version as a CDN fallback.

Console message: "jQuery is ready!"

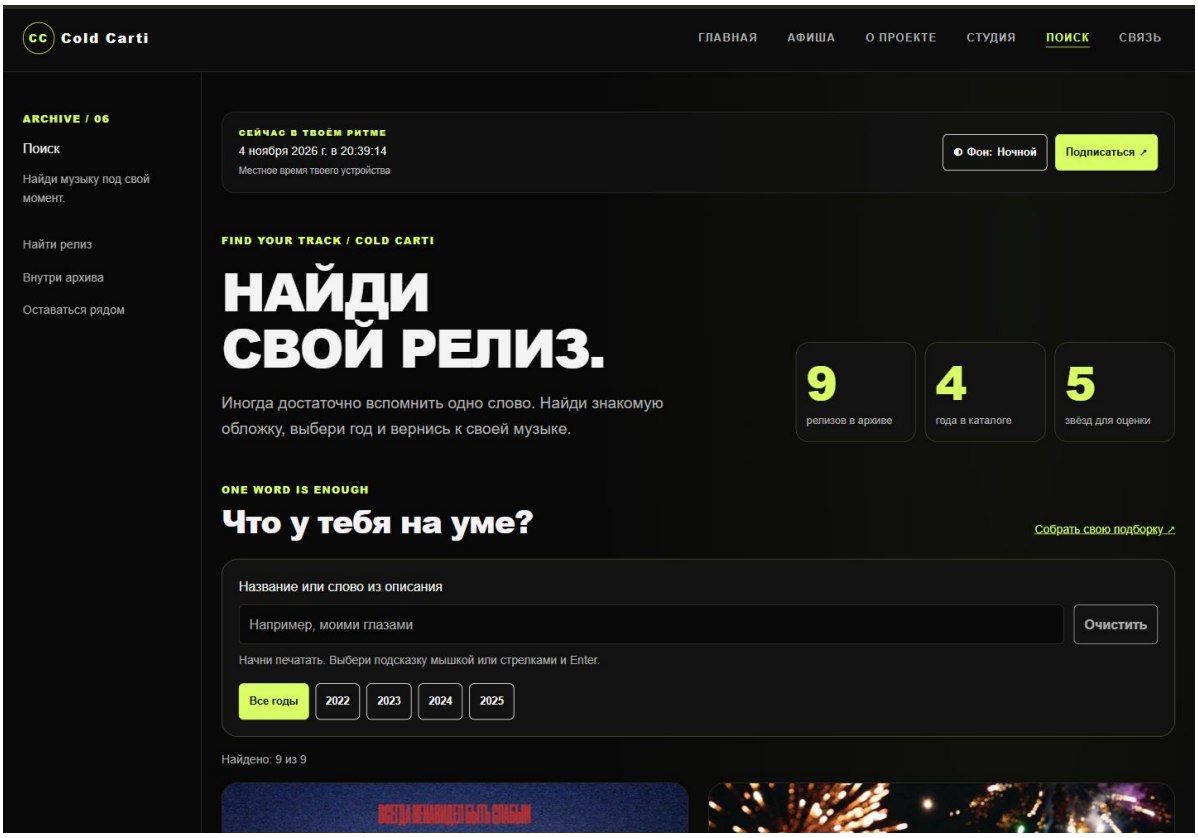
explore.html / lines 19-19

```
19        <script src="https://code.jquery.com/jquery-4.0.0.min.js" integrity="sha256-
  OaVG6prZf4v69dPg6PhVattBXkcOWQB62pdZ3ORyrao=" crossorigin="anonymous" defer></script>
```

script.js / lines 2-4

```
2 function initializeArchiveFeatures($) {
3     $(document).ready(function () {
4         console.log('jQuery is ready!');
```

Source: explore.html / lines 19-19; script.js / lines 2-4.



The new search page starts successfully; the browser console logged jQuery is ready!

Search and suggestions

Task 1. Real-time search and filter

1. Listen for input and read the query with .val().
2. Use .filter() to combine the search text and year, then update visible cards and the result count.

archive-search.js / lines 135-142

```
135         const $matches = $cards.filter(function matchesSearch() {
136             const $card = $(this);
137             const yearMatches = activeYear === 'all' || $card.data('year') ===
    activeYear;
138             return yearMatches && $card.data('searchText').includes(lowerQuery);
139         });
140         $cards.prop('hidden', true);
141         $matches.prop('hidden', false);
142         $count.text(`Найдено: ${$matches.length} из ${musicLibrary.catalogue.length}`);
```

Source: archive-search.js / lines 135-142.

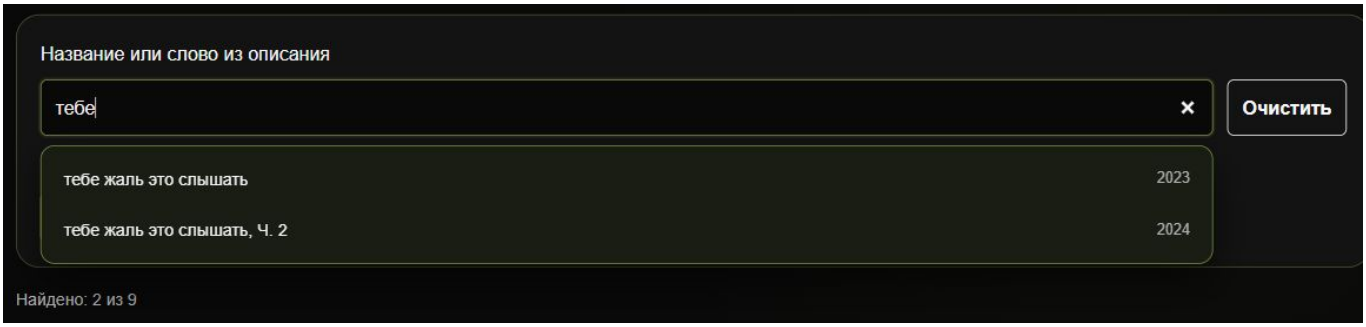
Task 2. Autocomplete suggestions

1. Find matching release titles and show up to five suggestions.
2. Choose an option with the mouse or arrow keys and Enter; Escape closes the list.

archive-search.js / lines 111-116

```
111         suggestions = musicLibrary.filterByYear(activeYear)
112             .filter((release) => normalize(release.title).includes(normalize(query)))
113             .slice(0, 5);
114         suggestions.forEach((release, index) => {
115             const $option = $('<li>', {
116                 class: 'archive-suggestion',
```

Source: archive-search.js / lines 111-116.



Result for tasks 1 and 2. Typing a title filters the archive without reloading. The dropdown suggests matching titles while typing.

Search highlighting

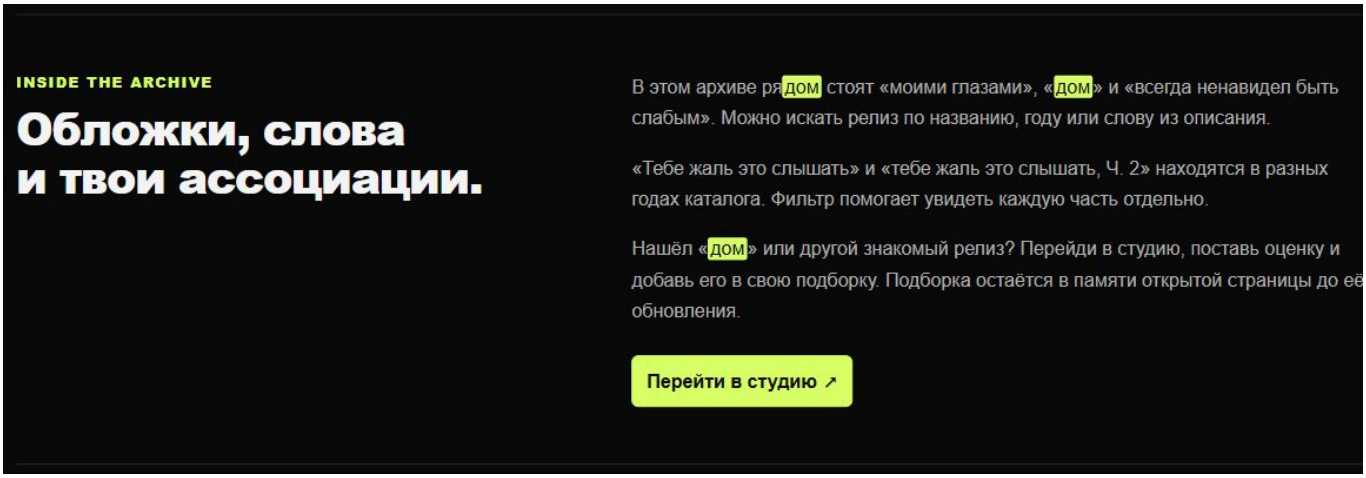
Task 3. Search highlighting

1. Find each literal occurrence in titles, descriptions and archive notes.
2. Insert safe text nodes and mark elements; clearing the query restores the original text.

archive-search.js / lines 43-51

```
43          while (match !== -1) {
44              $element.append(document.createTextNode(original.slice(position, match)));
45              $element.append($('<mark>', { class: 'archive-match' }).text(
46                  original.slice(match, match + query.length)
47              ));
48              position = match + query.length;
49              match = lowerText.indexOf(lowerQuery, position);
50          }
51          $element.append(document.createTextNode(original.slice(position)));
```

Source: archive-search.js / lines 43-51.



All matching occurrences are highlighted, including the archive notes.

Progress and counters

Task 4. Scroll progress bar

1. Divide the scroll position by the scrollable page height.
2. Update the gradient bar and its accessible value after scrolling, resizing or filtering.

script.js / lines 125-132

```
125             const scrollable = $(document).height() - $(window).height();
126             const progress = scrollable > 0
127                 ? Math.min(100, Math.max(0, $(window).scrollTop() / scrollable * 100))
128                 : 100;
129             $progress.attr('aria-valuenow', Math.round(progress));
130             $progress.find('[data-progress-fill]').css('transform', 'scaleX(' +
    progress / 100 + ')');
131             $('[data-progress-label]').text(Math.round(progress) + '%');
132         }
```

Source: script.js / lines 125-132.

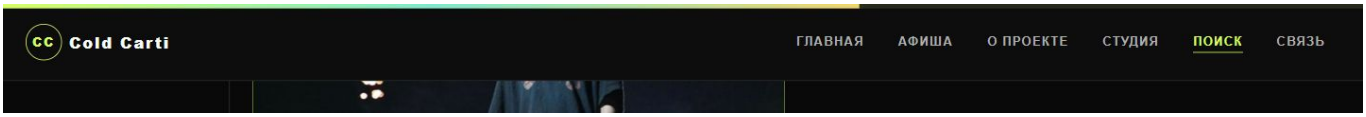
Task 5. Animated number counters

1. Start each counter once when it enters the viewport.
2. Use .animate() and .text() to count to 9 releases, 4 catalogue years and a 5-star rating scale.

script.js / lines 147-152

```
147                 $({ value: 0 }).animate({ value: target }, {
148                     duration: 900,
149                     step: function (value) { $counter.text(Math.floor(value)); },
150                     complete: function () { $counter.text(target); }
151                 });
152             });
```

Source: script.js / lines 147-152.



The bar above the navigation fills according to the actual scroll position.



The counters finish at the values used by this archive.

Form feedback

Task 6. Loading spinner on submit

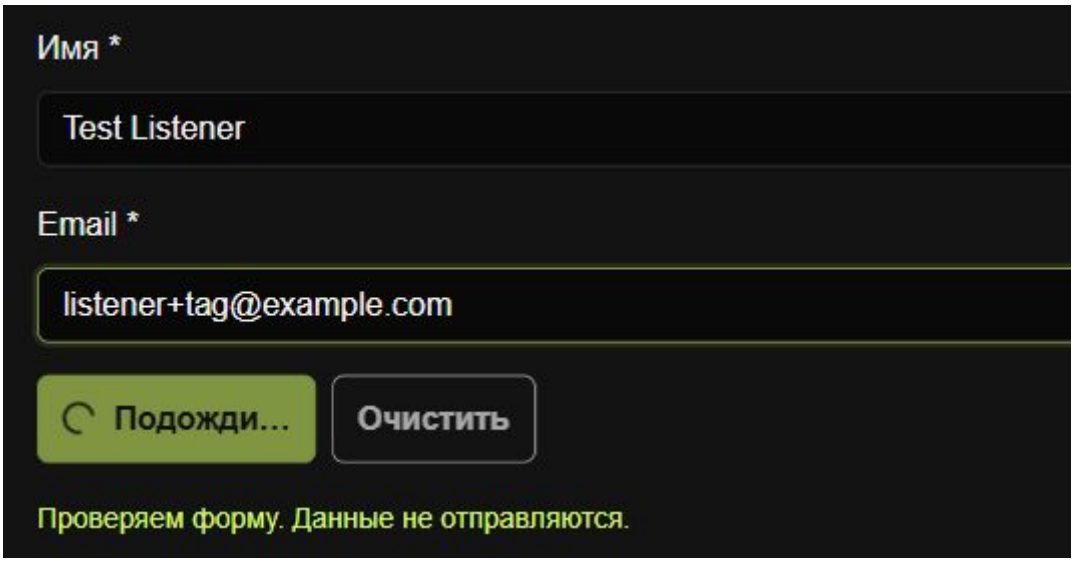
1. Run the existing validation first; only valid forms start the 1.6-second simulated check.

2. Show a spinner and disable buttons, then restore them. Editing or closing the dialog cancels the pending check.

script.js / lines 57-65

```
57            form.dataset.busy = 'true';
58            $form.attr('aria-busy', 'true');
59            $form.find('button').prop('disabled', true);
60            $button.empty().append(
61                $('<span>', { class: 'submit-spinner', 'aria-hidden': 'true' }),
62                document.createTextNode(' Подожди…')
63            );
64            $form.find('[data-form-status]').attr('data-state', 'pending')
65                .text('Проверяем форму. Данные не отправляются.');
```

Source: script.js / lines 57-65.



A valid form shows the loading state before its local result.

Notifications and copying

Task 7. Temporary notifications

1. Show relevant feedback after a filter, suggestion, copy or valid form action.
2. Use .stop(), .fadeIn() and .fadeOut() so repeated actions replace the previous notification.

script.js / lines 15-18

```
15            $toast.stop(true, true).text(message).fadeIn(reducedMotion.matches ? 0 :
   160);
16            toastTimer = setTimeout(function () {
17                $toast.fadeOut(reducedMotion.matches ? 0 : 350);
18            }, 2600);
```

Source: script.js / lines 15-18.

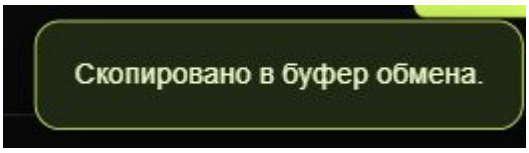
Task 8. Copy to clipboard

1. Read the title or public archive link from the clicked copy button.
2. After writeText() succeeds, show a checkmark and tooltip. A failed copy displays manual-copy feedback.

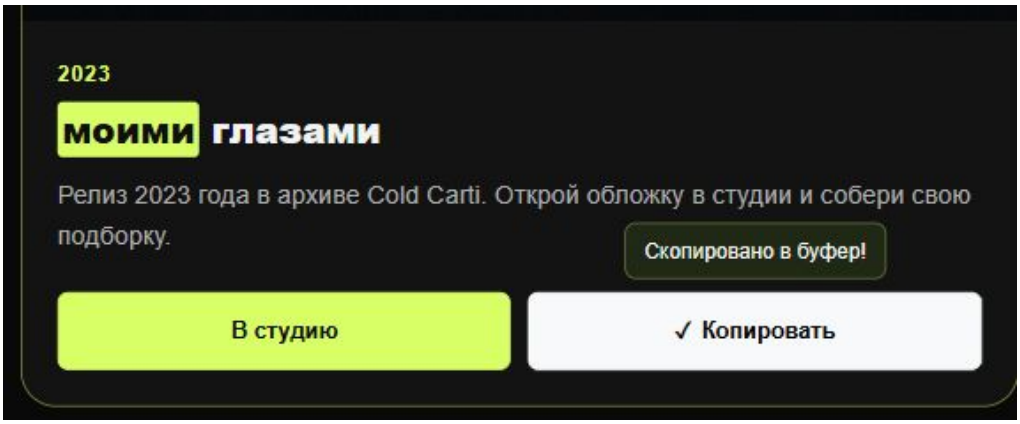
script.js / lines 99-107

```
99                await navigator.clipboard.writeText(text);
100               clearTimeout($button.data('copyTimer'));
101               $button.find('[data-copy-icon]').text('✓');
102               $button.find('.copy-tooltip').remove();
103               const tooltipId = 'copyTooltip-' + $('[data-copy-text]').index(this);
104               $('<span>', {
105                   class: 'copy-tooltip', id: tooltipId, role: 'tooltip',
106                   text: 'Скопировано в буфер!'
107               }).appendTo($button);
```

Source: script.js / lines 99-107.



A temporary notification confirms the completed copy action.



The title is copied; the button shows a checkmark and tooltip.

Loading images

Task 9. Lazy image loading

1. Start with a local placeholder and keep the cover path in data-src.
2. On scroll, use .offset() and the viewport bounds; assign the real src only when the image becomes visible.

script.js / lines 161-170

```
161                 const top = $image.offset().top;
162                 if (top >= viewportBottom || top + $image.outerHeight() <= viewportTop)
    return;
163                 const source = $image.attr('data-src');
164                 $image.one('load', function () {
165                     $image.addClass('is-loaded');
166                 }).one('error', function () {
167                     $image.addClass('image-unavailable');
168                 });
169                 $image.attr('src', source).removeAttr('data-src');
170             });
```

Source: script.js / lines 161-170.



The cover is loaded when its card enters the viewport.

Checks and reflection

- JavaScript syntax checks and the static website checker passed.

- All 14 existing model and email tests passed.

- Chromium: all seven pages fit at 320, 390, 768, 992 and 1440 pixels (35 layouts).

- Checked year and text search, suggestions by mouse and keyboard, literal symbols and clearing highlights.

- Copied a title and checked its clipboard value, checkmark and tooltip.

- Checked invalid emails, submit loading, completion and cancellation after editing or closing a dialog.

- A controlled CDN failure loaded the local jQuery copy. Previous FAQ, background, keyboard menu and Studio controls still worked.

Challenges and solutions

Search text can contain HTML or pattern symbols. Build highlights from text nodes instead of treating the query as markup.

A form can change during the simulated delay. Cancel its timer and restore the original buttons when the user edits it.

Filtering moves cards on the page. Recheck image visibility and scroll progress after the results change.

Reflection

jQuery connects the existing catalogue to search and feedback with short DOM methods. Data and validation remain shared. Forms simulate a check without sending or saving data.

Website

https://kanatovs.github.io/music-bandv3/explore.html

Project: https://github.com/kanatovs/music-bandv3

Prepared on 4 November 2026.